

by Kubernetes—and forwards them to Logstash or Elasticsearch. Filebeat is configured to append metadata such as pod name, namespace, and labels to each log event.

- **Logstash:** Acts as the processing layer. It performs advanced parsing, such as extracting fields from structured JSON logs or enriching events with custom tags. This makes downstream filtering and alerting more precise.
- **Elasticsearch:** Stores all log data in a structured, searchable format. It supports distributed indexing, high availability, and powerful query capabilities—ideal for multi-service environments.
- **Kibana:** Provides a visual layer over Elasticsearch. We use Kibana to build dashboards that track error rates, usage patterns, and service-specific metrics, helping us spot trends and anomalies in real time.

Log ingestion workflow

Our logging flow integrates cleanly with Docker and Kubernetes, using standard logging streams and metadata tagging.

- **Application output:** All backend services (Spring Boot microservices) and the frontend (React, served via Nginx or Node.js) are configured to log to stdout and stderr.
- **Container log capture:** Kubernetes automatically collects these logs from running containers and makes them available at the node level.
- **Filebeat collection:** Filebeat DaemonSets on each node read these logs and enrich them with Kubernetes-specific metadata. The configuration includes module-level filters to tag logs by service, severity, and pod identity.
- **Log processing:** Logs are forwarded to Logstash for structured parsing. Depending on the service, Logstash may apply JSON decoders, Grok patterns, or timestamp normalisation before forwarding data to Elasticsearch.
- **Centralised storage:** Once indexed, logs are available for search and analysis in Elasticsearch. This setup supports Kibana queries and visualisations without requiring any direct access to running pods.

Kubernetes deployment via Ansible Playbook

The full source code for the Ansible playbook used to deploy HealSphere is available at https://github.com/Kanan-30/HealSphere_Project/blob/main/deploy-k8s.yml. This playbook is executed as part of the Jenkins pipeline and automates the deployment of Kubernetes manifests for all backend and frontend services.

Playbook overview

- **Name:** Deploy HealSphere App to Kubernetes.
- **Target Host:** localhost – The tasks are executed on the same machine where the playbook runs, typically the Jenkins agent.
- **gather_facts:** Set to false, since no system facts are

required for Kubernetes operations.

- **Connection:** local – Commands are executed locally without SSHing into remote nodes.

Variables

- **k8s_manifest_path:** Defines the location of the Kubernetes manifests. Built using the Jenkins \$WORKSPACE environment variable appended with `/MindNotes/k8s`.
- **k8s_namespace:** The deployment namespace, defaulted to `mindnotes`. This can be overridden at runtime using Jenkins parameters, allowing flexibility for staging or production.

Here's an explanation of the tasks.

Task 1: Apply Kubernetes manifests

- **Purpose:** Deploy all YAML files in the specified directory to the target Kubernetes namespace.
- **Execution:** `kubectl apply -n <namespace> -f <manifest-path>`
- **Result handling:**
 - Output is captured in a variable named `kubectl_apply_result`.
 - The task is marked as 'changed' if output includes 'created' or 'configured', indicating new or updated resources.
 - The task is marked as 'failed' if the command exits with a non-zero return code (`rc != 0`).

Task 2: Display kubectl output

- **Purpose:** Show detailed output from the previous task for visibility and debugging.
- **Implementation:**
 - Uses `ansible.builtin.debug` to print the command output.
 - Set to verbosity: 1, so it only appears when Ansible is run in verbose mode (e.g., `ansible-playbook -v`), keeping standard output clean.

Post-build actions in Jenkins

The Jenkins pipeline is configured to send status notifications based on the outcome of the Ansible deployment.

- **On success:** Sends an email to `kanan.gupta@iiitb.ac.in` with the subject: '[K8s] HealSphere Deployment SUCCESS' and a message confirming successful rollout.
- **On failure:** Sends an email with the subject: '[K8s] HealSphere Deployment FAILED' including a direct link to the Jenkins console output for quick debugging.
- **Always:** The workspace is cleaned at the end of every build to ensure no residual files are left behind. This helps maintain a consistent and isolated build environment.

Security and secrets management

Security considerations were integrated throughout the stack both at the application and infrastructure levels.